

AI & LLM FUNDAMENTALS FOR SOFTWARE DEVELOPERS

Advanced Prompt Engineering

From single prompts to orchestrated programs.

SHIP IT 3 live demos • 7 take-home notebooks • spin up [MLflow](#) :5000 and code along.

- 00 Opening
- 01 The Map
- 02 Primary Techniques
- 03 Secondary Techniques
- 04 Ladder & Composition
- 05 Reasoning Models
- 06 Production
- 07 Closing

The Session 5 ceiling

Five reasoning chains. All voted the same. All wrong.



✗ SELF-CONSISTENCY N=5 → UNANIMOUS → WRONG

Real RCA: 5xx spike, 14:02:17 UTC

CoT + Self-Consistency, N=5

→ all 5 chains vote “**deploy rollback**”

GROUND TRUTH

DB connection-pool exhaustion

Deploy was 9 minutes earlier — irrelevant.

SHIP IT Self-Consistency averages noise — it can't vote its way out of a shared bad assumption. Today we add **search, grounding, decomposition, verification**.

A prompt is no longer a string

A prompt is now a control-flow program.

SESSION 5

A single LLM call

- prompt → completion
- one pass
- anatomy • CoT • self-consistency

SESSION 6

A multi-call program

- DAG: 4 LLMs + 2 tools + 1 validator
- routed • evaluated • retried
- per-step pass-rate • DLQ • observability

SHIP IT Same language, different semantics. Every Session 5 lesson still applies — now to **each sub-prompt** in the program (its own anatomy, validator, eval, cache key, injection defence).

Five families

Learn the patterns, not the 24 names.

Decomposition

split the problem

Chaining

Least-to-Most

Plan-and-Solve

SoT

Search / Branch

externalise the search

ToT

GoT

Maieutic

Self-Correction

*verify own work**

Self-Refine

Reflexion

Constitutional

External Grounding

use a real tool

ReAct

PoT

Gen-Knowledge

Step-Back

Exemplar / Context

shape the input

Auto-CoT

Analogical

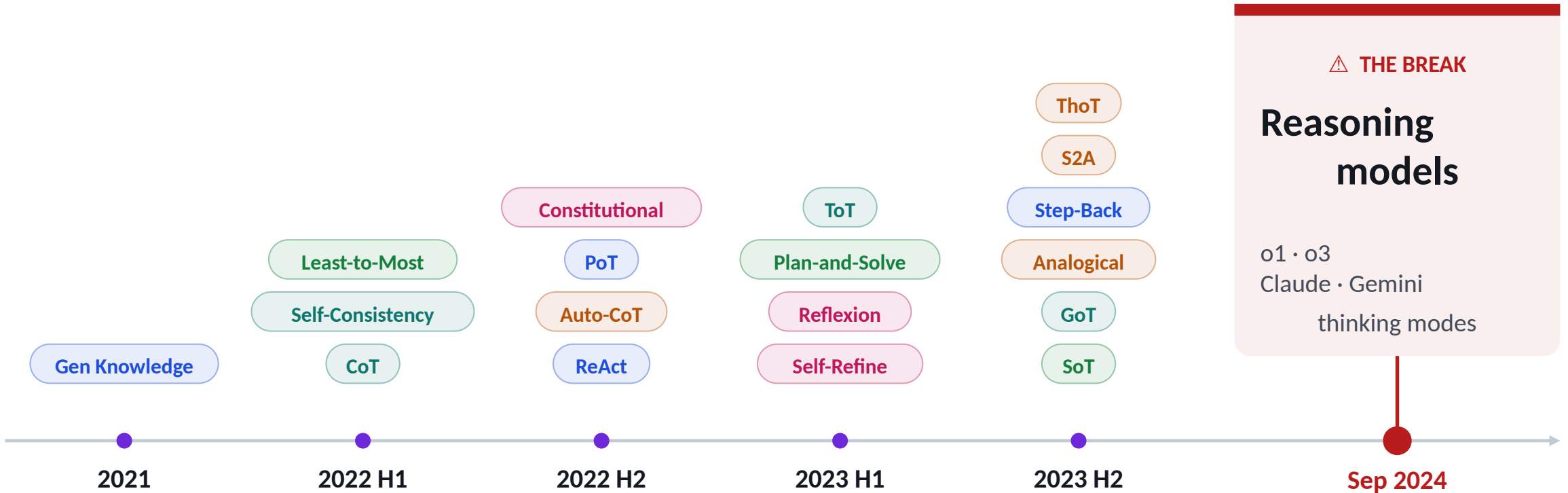
S2A

ThoT

SHIP IT Two decision questions carry the whole session: **which family fits this failure mode?** and **do I have a verifier?** (*verify own work needs an external signal — slide 25*)

Five years, one inflection point

Half of these get absorbed into reasoning models in Sep 2024.



Tree of Thoughts: the 4% → 74% slide

CoT is one straight line. ToT branches, scores, and searches.

4%

CoT — one chain, commit early



74%

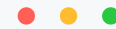
ToT — propose · evaluate · search

Game-of-24
GPT-4
Yao et al. 2023
arXiv:2305.10601

READ WITH AN ASTERISK

4 → 74 needs **catastrophic early-commit** + a **cheap, reliable evaluator**.

Most production tasks meet **neither** — so the headline rarely transfers.

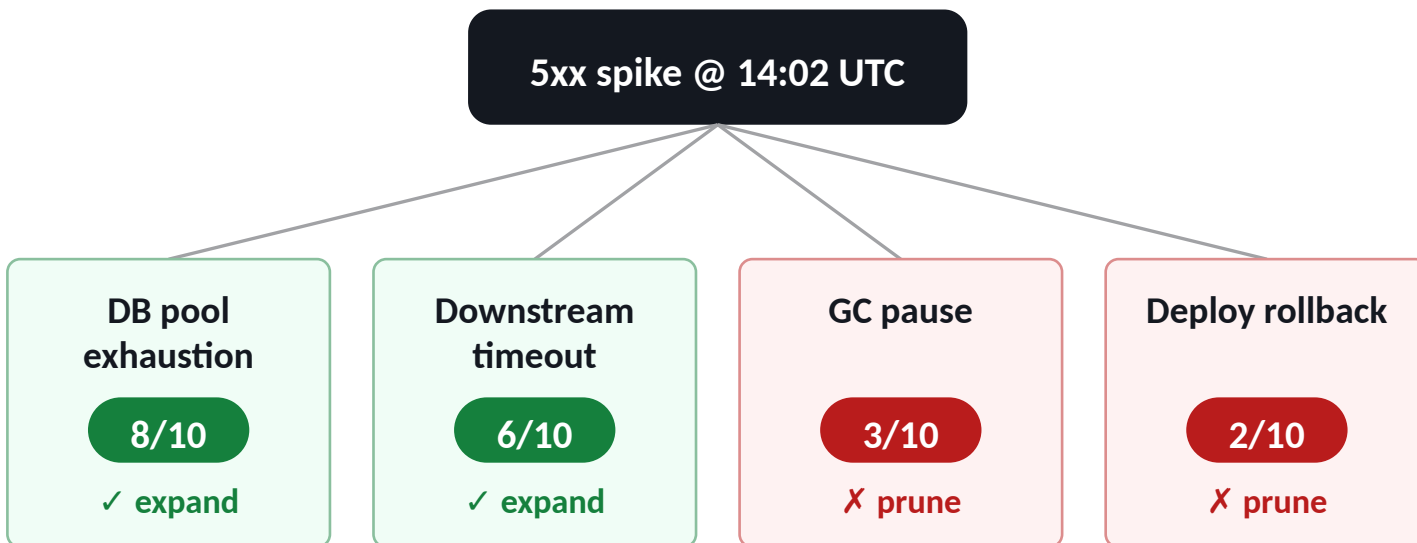


tree-of-thoughts (BFS)

```
def tot_bfs(state, propose, evaluate, beam=3, depth=4):
    frontier = [state]
    for _ in range(depth):
        kids = [c for s in frontier for c in propose(s)]
        frontier = sorted(kids, key=evaluate,
                          reverse=True)[:beam]
    return frontier[0]
```

ToT for incident root-cause analysis

Four hypotheses. Two expand, two prune — by score.



USE WHEN

- clear intermediate states
- cheap, reliable evaluator
- tolerant of 30–50× tokens

DON'T USE WHEN

- value function is unreliable (search amplifies bad evaluators)
- you're on a reasoning model (paying twice)
- real-time UX

Contrast: Self-Consistency voted 5/5 for *deploy rollback* — ToT's evaluator prunes it at 2/10.

Prompt chaining: the 62% accuracy floor

$0.95 \times 0.92 \times 0.88 \times 0.80 = 62\%$. Chains decay fast.



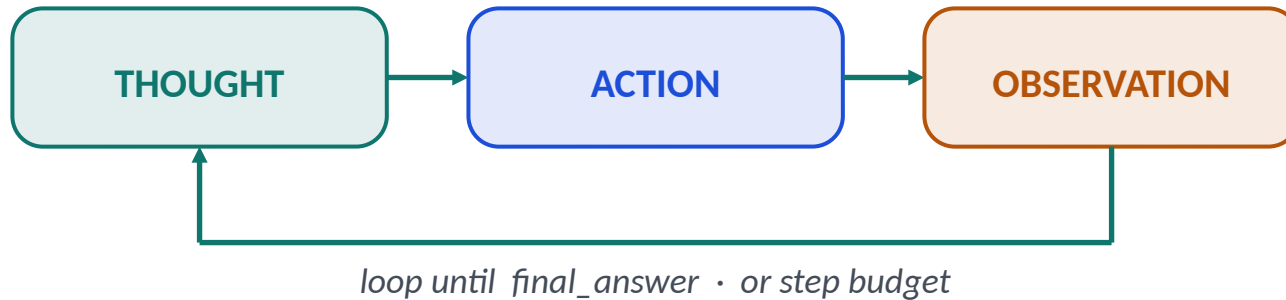
validate + retry-with-feedback

```
@mlflow.trace(span_type=SpanType.LLM)
def run_step(prompt, schema, max_retries=2):
    for _ in range(max_retries + 1):
        out = llm(prompt)
        try:
            return schema.model_validate_json(out)
        except ValidationError as e:
            prompt += f'Failed: {e}. Fix and retry.'
```

- Chain accuracy = **product** of step accuracies.
- **Per-step pass-rate** finds the weak step — fix only that one.
- Pydantic validator + retry recovers **70–80%** of bad JSON in one retry.

ReAct: Thought / Action / Observation

A REPL where the LLM is the developer.



one turn = one LLM call

```
T1  thought: check the deploy log
T1  action : tail_logs(api, lines=200)
T1  obs    : [connection pool exhausted]
T2  thought: confirm with pool metrics ...
```

PRODUCTION HARDENING — the 3 that matter

- **Cycle detection** — hash (`tool`, `args`), break on repeat
- **Error cap** — 3 failures in a row → bail
- **Max-steps cap** — 10-15 turns, then fall back to CoT

ALFWorld +34 pp · WebShop +10 pp · Yao 2022 · arXiv:2210.03629

USE WHEN

tool use is the constraint ·
multi-step exploration ·
cycle/error handling OK

DON'T USE WHEN

read-only computation
(use PoT) · latency <500
ms · reasoning model with
native tools

Generate Knowledge: self-RAG (use sparingly)

Two calls: generate the facts, then apply them.

STEP 1 — abstract the knowledge



```
# AWS SDK v2 → v3 deprecations
AWS.S3()          -> new S3Client()
s3.upload(params) -> PutObjectCommand
.promise()        -> await send(cmd)
```

STEP 2 — apply to the code

```
- const s3 = new AWS.S3();
+ const s3 = new S3Client({});
- await s3.upload(p).promise();
+ await s3.send(new PutObjectCommand(p));
```

⚠ **Amplifies hallucinations** when the model's parametric knowledge is wrong. **Never** use for ground-truth queries — pretty hallucinations are still hallucinations.

Pick this when you...	Mechanism	Best for
Step-Back	abstract the principle	STEM, multi-hop QA
Generated Knowledge	abstract the facts	bridging via training data
RAG (Session 8)	retrieve from a store	ground-truth required

Live demos — code along with us

Three live, four to take home. All self-contained.

► LIVE NOW

D1

Tree of Thoughts

CoT 4% → ToT 74%

D2

Bug-triage chain

Pydantic + retry + registry

D3

MLflow eval

compare 4 techniques

○ OPTIONAL — take-home self-study

D4

ReAct K8s bot

D5

PoT vs CoT

D6

Self-Refine + ruff

D7

SoT parallel

SHIP IT Set 3 env vars and run: `MLFLOW_TRACKING_URI • LLM_API_KEY • LLM_MODEL`. No key? **mock mode** still demonstrates the structure. Every prompt + response is captured in MLflow.

Plan-and-Solve: the free upgrade

Two free sentences. Ship them everywhere this week.

BEFORE — zero-shot CoT

“Let's think step by step.”

AFTER — Plan-and-Solve (PS+)

*“First understand the problem and devise a plan.
Then execute it step by step, paying attention to
variable values.”*

+9.1 pp

arithmetic avg · Wang 2023 ·
2305.04091

USE WHEN

- any zero-shot CoT prompt — pure drop-in, no examples, no extra calls
- PS+ variant names variable values → another **+1-2 pp**

Abstract-then-apply: pick by what's abstracted

Three techniques, identical 2-call shape.



Technique	Abstracts	Best for	Don't when
Step-Back	the principle	STEM, multi-hop QA • K8s debug	abstraction picks wrong category
Generated Knowledge	the facts	API deprecation review	parametric knowledge is wrong
Least-to-Most	the subproblems	multi-table SQL • SCAN 16%→99.7%	bad decomposition cascades

SHIP IT Diagnose by failure mode: **wrong category?** → Step-Back. **Missing background?** → GenKnowledge. **Won't fit one call?** → Least-to-Most.

PoT + Self-Refine: execute, then iterate

Don't let the LLM do mental arithmetic. Don't let the critic float.

PROGRAM-OF-THOUGHT

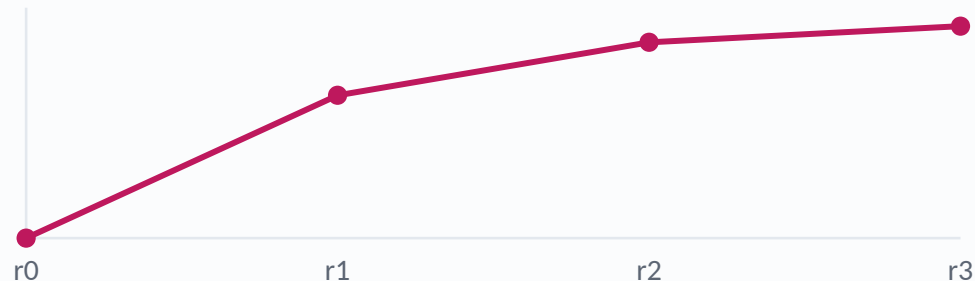


```
code = llm(f'Write Python. print(result).\n{q}')
return safe_exec(extract_code(code),
                 timeout=10) # sandboxed
```

GSM8K +8.5 pp • FinQA +24.1 pp • Chen 2022

SELF-REFINE — gen • critique • refine

round 1 = big lift • cap at 2 rounds



⚠ THE PRINCIPLE: GROUND THE MODEL WITH SOMETHING DETERMINISTIC

PoT grounds with a Python **interpreter**. Self-Refine must ground with a **linter** / **test runner** / **rubric** — anything the model didn't already see.

Self-Refine with **no external verifier degrades performance** (slide 25). Skip both if a reasoning model does interpreter tool-calls natively.

Three structure tricks worth knowing

Each is one wiki page — here in one chart.

Skeleton-of-Thought

2.39× wall-clock

Skeleton first, then fan out sections in parallel — same tokens.

USE WHEN

list-shaped outputs: architecture review, multi-finding code review

Auto-CoT

zero labelling

k-means cluster questions, auto-generate the rationales.

USE WHEN

building a new prompt library from scratch

Contrastive CoT

+15 pp w/ SC

Show positive AND negative exemplars of a known error.

USE WHEN

a specific, repeatable bug pattern in production

Awareness wall: 10 more techniques

Name-recognition only — the wiki covers each in ~50 lines.

GoT

ThoT

Analogical

S2A

EmotionPrompt

Reflexion

Constitutional

Maieutic

Complexity-based

Faithful CoT

REFLEXION — worth the name-drop

Self-Refine + **memory across trials**. HumanEval 80 → 91%.

Real production use case — but **needs an external verifier**.

EMOTIONPROMPT — the surprising one

“This is important to my career.” **+115%** on BIG-Bench Hard in 2023.

Neutralised on heavy-RLHF frontier models — re-eval after every model bump.

The 11-rung complexity ladder

Climb only when the current rung's eval plateaus.

10 Switch to reasoning model — CoT/ToT/PnS now redundant

9 Compose a pipeline DAG

8 Search — ToT (beam 2-3) or Reflexion

7 Self-correct — Self-Refine or Constitutional

6 External grounding — PoT or ReAct

5 Decompose — Least-to-Most or Chain

4 Plan-and-Solve (the free upgrade)

0-3 Session 5 — Zero-shot • ZS-CoT • Few-shot CoT • SC (N=5)

The rule that matters:

Each rung adds variance, tokens, failure modes. Moving up without eval evidence is anti-engineering.

DECISION SHORTCUTS

Numeric task?

→ jump to Rung 6 — PoT

Needs current data?

→ jump to Rung 6 — ReAct

Latency-OK + budget?

→ jump to Rung 10 — reasoning model

Six compounds that work • five that don't

Recognise the compounds. Name the anti-patterns.

✓ PAIR WELL — compose freely

Plan-and-Solve + PoT

plan in prose, execute in code

ReAct + Self-Refine

act to gather evidence, then refine

Gen-Knowledge + Step-Back

recall facts, then abstract

SC + Complexity-based

vote among long chains (+15 pp)

ToT + PoT (per-leaf)

numeric value fn via interpreter

SoT + Constitutional

parallel sections, lint each

✗ PAIR BADLY — skip these

ToT + Self-Refine (same trial)

refining branches you may prune

Reflexion + Self-Refine

double critique, no grounding

EmotionPrompt + reasoning model

neutralised, wasted tokens

ToT + reasoning model

paying twice for the search

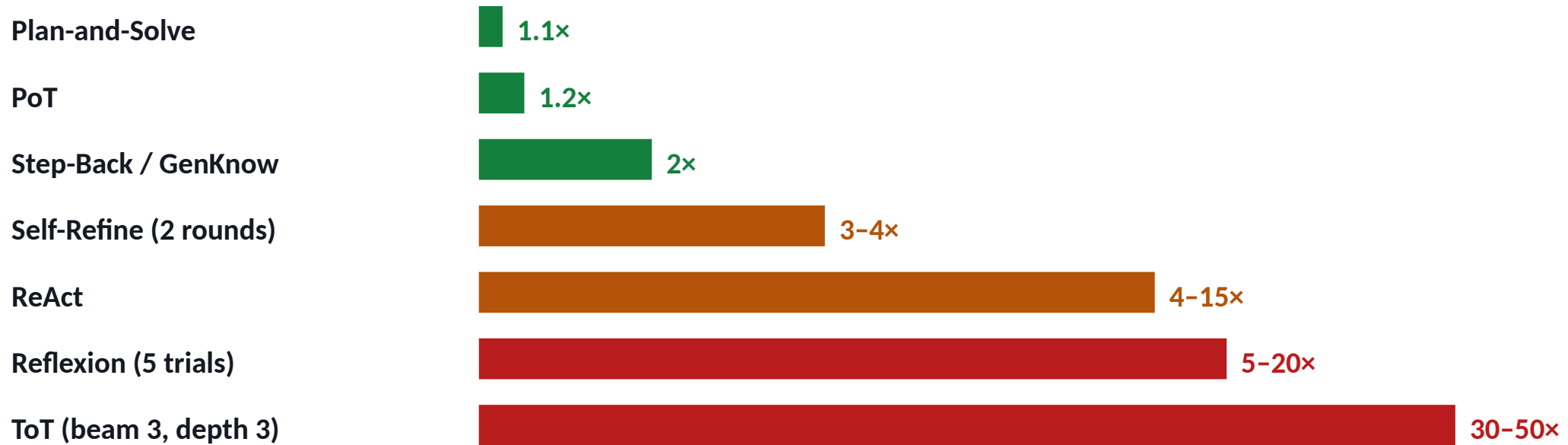
S2A + clean short context

rewrite overhead > noise saved

SHIP IT Ablation discipline: take a compound in your stack, disable one half, re-measure on the eval set. If accuracy doesn't move, the disabled half is ceremony.

Cost lookup (× a single CoT call)

The token-cost spread across techniques is ~100×



Skeleton-of-Thought: ~1× tokens but **N+1 parallel calls** — same cost, saves wall-clock (not a cost play, a latency play).

SHIP IT When MLflow shows ToT at **\$0.30/call** vs PnS at **\$0.005**, the architecture decision is settled by the number, not the opinion.

Reasoning models: read the guidance exactly

Avoid CoT prompts. Asking it to reason more can hurt.

“

“Avoid chain-of-thought prompts — since these models reason internally, prompting them to ‘think step by step’ is unnecessary.”

“

“Asking a reasoning model to reason more may actually hurt the performance.”

— OpenAI • official o1 / o3 prompting guidance • use the `developer` role on o1-2024-12-17+



o-series — set effort, not CoT

```
client.chat.completions.create(  
  model='o3',  
  reasoning_effort='medium',    # 'low' | 'medium' | 'high'  
  messages=[{'role': 'developer', 'content': 'You are a senior SRE.'},  
             {'role': 'user', 'content': query}],  
)
```

Claude Extended Thinking • Gemini budget

Every vendor exposes a budget knob. Use it.

CLAUDE EXTENDED THINKING

```
client.messages.create(  
    model='claude-sonnet-4-6',  
    thinking={'type': 'enabled',  
              'budget_tokens': 8000},  
    messages=[{'role': 'user', 'content': query}],  
)  
# `think` tool: +54% pass^1 (Tau-Bench airline)
```

GEMINI — two APIs, don't mix

```
# Gemini 2.5 — numeric budget  
{  
  'thinking_config':  
    {'thinking_budget': -1} # 0=off, -1=dynamic  
}  
  
# Gemini 3 — categorical (recommended)  
{  
  'thinking_level': 'high' # low | medium | high  
}
```

⚠ COMPATIBILITY FOOTGUN

Never pass OpenAI `reasoning_effort` and Gemini `thinking_level` through the same OpenAI-compatible adapter — it picks one or errors.

What dies • what survives

Drop CoT/ToT/PnS. Keep ReAct/PoT/Reflexion/Decomposition.

REDUNDANT — the model does this in its thinking phase

- ~~✗ Zero-shot CoT~~
- ~~✗ Plan-and-Solve~~
- ~~✗ Tree of Thoughts~~
- ~~✗ Auto-CoT exemplars~~
- ~~✗ Self-Consistency (offline tasks)~~

STILL VALUABLE — orthogonal to reasoning

- ✓ Decomposition / Chaining — task structure is your job
- ✓ ReAct — tool use is orthogonal
- ✓ PoT — interpreters still beat LLM arithmetic
- ✓ Reflexion — cross-trial memory the model lacks
- ✓ Constitutional • S2A • ThoT — context-cleaning
- ✓ Skeleton-of-Thought — latency is latency

The doubled-cost trap: Wharton GAIL 2025 — o3-mini + explicit CoT = **+2.9% at 20–80% more tokens**. Hand-engineering reasoning on a reasoning model pays twice for worse results.

MLflow Tracing 3.10+ — every call captured

Five lines turn any chain or agent into a debuggable system.

1 • Decorator

```
@mlflow.trace(name=...,  
              span_type=SpanType.LLM)
```

2 • Nested spans

```
with mlflow.start_span(name=) as sp:  
    sp.set_inputs/outputs(...)
```

3 • Autolog

```
mlflow.openai.autolog()  
mlflow.anthropic.autolog()
```

4 • Annotate

```
mlflow.update_current_trace(  
    tags={'cost_usd': '0.03'})
```



Prompt Registry — version every chain step

```
pv = mlflow.genai.register_prompt(name='trriage_extract', template=T, commit_message='v3 fix')  
EXTRACT = mlflow.genai.load_prompt(f'prompts:/trriage_extract/{pv.version}')
```

mlflow.genai.evaluate — compare on one eval set

Built-in scorers + @scorer. The hand-rolled eval loop is gone.

Pattern 2 — the production eval workflow

```
from mlflow.genai.scorers import Correctness, Guidelines, Safety, scorer

TECHNIQUES = {'zero_shot_cot': zsc, 'plan_and_solve': pns, 'tot': tot}

@scorer
def cites_evidence(outputs: str) -> bool:
    return bool(re.search(r'\d{2}:\d{2}', outputs)) # timestamp

for name, fn in TECHNIQUES.items():
    with mlflow.start_run(run_name=name):
        mlflow.genai.evaluate(data=eval_set, predict_fn=fn,
                              scorers=[Correctness(), Safety(), cites_evidence])
```

WHAT YOU GET

- **12 lines** replace any bespoke eval loop.
- Built-ins: `Correctness` · `Guidelines` · `Safety` · `RelevanceToQuery` · `Completeness`.
- `@scorer` turns any boolean assertion into a metric.
- Custom scorers read trace tags (`cost_usd`, `latency`) — make **cost a first-class metric**.
- Compare runs in MLflow; highest Correctness at lowest cost **ships**.

Misconception 0: self-correction needs a verifier

Self-correction without an external verifier degrades accuracy.

Pure “do you think this is right?” loops are not useless — they are harmful.



WITHOUT verifier

–3 pp on GSM8K — the model talks to a mirror



WITH verifier (ruff + tests)

accuracy climbs — grounded in an external signal

Huang et al. 2023 · arXiv:[2310.01798](#) — *LLMs Cannot Self-Correct Reasoning Yet*

Affected: Self-Refine · Constitutional critique · Reflexion · ToT value functions — *all need an external signal*

Acceptable verifiers: rubric doc · unit-test runner · linter · ground-truth reference · executor · retrieval

Verify in practice: disable the critic, re-run. If Correctness/mean doesn't improve ≥ 2 pp on >100 cases, strip the loop.

Practitioner wisdom: the 6 maxims

Memorise before climbing past Rung 4.

1 • Eval set > fancier technique

The eval is the deliverable; technique is implementation.

2 • Schema-validate every JSON

Pydantic at every boundary. Missing key in step 3 = silent failure in step 7.

3 • Cache at temp=0

LLMs are near-deterministic — free correctness, free latency.

4 • Per-step pass-rate

Find the weak step before changing any prompt.

5 • Re-run evals every model bump

Floating aliases are silent bombs.

6 • Cache yes / no

Chain steps & read-only tools = YES. Self-Refine intermediates = NO.

Three questions

If you remember only this, you have 80% of the value.

1

Which family fits this failure mode?

Decomposition • Search • Self-Correct •
Grounding • Exemplar

2

Do I have a verifier?

Huang 2310.01798 — without one, self-
correct is a noise generator

3

Am I paying twice on a reasoning model?

Drop CoT/ToT/Plan-and-Solve on o1 /
o3 / Claude-thinking

Session 7 preview • Q & A

Next: turn these pipelines into production apps.

Session 7 — Application Development & Tool Integration

- Tool schemas — OpenAI function calling, Anthropic tool use, MCP
- FastAPI endpoints wrapping chains and agents
- End-to-end observability — MLflow Tracing across HTTP boundaries
- Cost budgeting and rate-limit defence

Q & A

TO GET US STARTED

“Who's running Self-Refine without an external verifier?”

“Who's using ToT on o3 today?”

Benchmarks at a glance

Every number in this deck, sourced.

Technique	Paper • arXiv	Headline result
ToT	Yao 2023 • 2305.10601	Game-of-24 GPT-4: 4% → 74%
Least-to-Most	Zhou 2022 • 2205.10625	SCAN: 16.2% → 99.7%
ReAct	Yao 2022 • 2210.03629	ALFWorld: +34 pp
Reflexion	Shinn 2023 • 2303.11366	HumanEval pass@1: 80% → 91%
Self-Refine	Madaan 2023 • 2303.17651	7-task avg: +20% (needs verifier)
Program-of-Thought	Chen 2022 • 2211.12588	FinQA: 40.4% → 64.5% (+24.1)
Step-Back	Zheng 2023 • 2310.06117	TimeQA: +27 pp
Skeleton-of-Thought	Ning 2024 • 2307.15337	12 LLMs: 2.39× speedup
Plan-and-Solve	Wang 2023 • 2305.04091	Arithmetic avg: +9.1 pp
Anthropic think tool	Anthropic 2024	Tau-Bench airline: +54% pass¹
Self-correction caveat	Huang 2023 • 2310.01798	w/o verifier *degrades*

The three live demos (for self-study)

All self-contained. Mock mode if you have no API key.

D1

ToT on Game-of-24

[demo_01_tot_game_of_24.ipynb](#)

CoT 4% → ToT 74% reveal · ~5 min

D2

Bug-triage chain

[demo_02_bug_triage_chain.ipynb](#)

3-step Pydantic + retry + registry · ~6 min

D3

MLflow technique compare

[demo_03_mlflow_comparison.ipynb](#)

genai.evaluate on 4 techniques · ~5 min

Optional take-home: D4 ReAct K8s bot · D5 PoT vs CoT finance · D6 Self-Refine + ruff/mypy · D7 SoT 4× parallel

```
SHIP IT pip install -r requirements.txt · export MLFLOW_TRACKING_URI=... LLM_API_KEY=... LLM_MODEL=... ·  
jupyter lab
```